Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Леваков Н.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 29.09.26

Москва, 2026

Постановка задачи

Вариант 19.

Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процесс child1 и child2 производят работу над строками. Процессы пишут результаты своей работы в стандартный вывод

Правило фильтрации: с вероятностью 80% строки отправляются в pipe1, иначе в pipe2.
Дочерние процессы удаляют все гласные из строк

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- int pipe(int *fd); – создает канал.
- int dup2(int oldfd, int newfd); - копирует дескриптор oldfd в newfd, используется для перенаправления stdin на pipe.
- int execv(const char *path, char *const argv[]); - заменяет текущий процесс на новую программу.
- int open(const char *path, int flags, mode_t mode); - открывает файл. Флаги: O_RDONLY, O_WRONLY, O_CREAT, O_TRUNC.
- ssize_t read(int fd, void *buf, size_t count); - читает данные из дескриптора.
- ssize_t write(int fd, const void *buf, size_t count); - пишет данные в дескриптор.
- int close(int fd); - закрывает дескриптор.
- pid_t wait(int *status); - ждет завершения дочернего процесса.
- ssize_t readlink(const char *path, char *buf, size_t bufsize); - читает содержимое символической ссылки.

Пользователь запускает программу:

./parent input.txt file1.txt file2.txt

Затем родитель открывает input.txt, после этого он создает child1 и child2 через fork и execv. Дети перенаправляют свой stdin на чтение из pipe через dup2. Родитель читает из input.txt и фильтрует строки (80% в первый, 20% во второй). Дети читают из pipe, удаляют гласные, пишут в свои файлы. Родитель закрывает pipe и ждет детей через wait.

Алгоритм фильтрации строк в родителе решает в какой канал отправляется строка. Реализован он по следующему принципу: мы используем генератор псевдослучайных чисел rand(). Перед первым вызовом rand() генератор инициализируется через srand(time(NULL)), чтобы последовательность чисел была разной при каждом запуске. Для каждой прочитанной порции данных вычисляется rand() % 100, что дает нам число от 0 до 99. Если число меньше 80, то оно попадает в pipe1, иначе в pipe2.

Алгоритм удаления гласных реализован с помощью функции is_vowel(), которая возвращает 1, если символ гласная и 0, если согласная. В цикле перебираются все символы прочитанной строки, если символ не гласная, то копируем его в буфер out, а если гласная – пропускаем, результат записываем в файл.

Код программы

parent.c

```c
#include <unistd.h>
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <fcntl.h>

int main(int argc, char **argv) {
    if (argc < 4) {
        write(2, "Usage: ./parent <input.txt> <file1.txt> <file2.txt>\n", 49);
        exit(1);
    }

    srand(time(NULL));

    char progpath[1024];
    ssize_t len = readlink("/proc/self/exe", progpath, sizeof(progpath) - 1);
    if (len == -1) {
        write(2, "Error: readlink\n", 16);
        exit(1);
    }

    while (progpath[len] != '/') { --len; }
    progpath[len] = '\0';

    int input_fd = open(argv[1], O_RDONLY);
    if (input_fd == -1) {
        write(2, "Error: open input\n", 18);
```

```
        exit(1);
    }

    char *file1 = argv[2];
    char *file2 = argv[3];

    int pipe1[2], pipe2[2];
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        write(2, "Error: pipe\n", 12);
        exit(1);
    }

    pid_t pid1 = fork();
    if (pid1 == -1) {
        write(2, "Error: fork child1\n", 19);
        exit(1);
    }

    if (pid1 == 0) {
        dup2(pipe1[0], 0);
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        char path[2048];
        snprintf(path, sizeof(path) - 1, "%s/child", progpath);

        char *args[] = {"./child", file1, NULL};
        execv(path, args);
        write(2, "Error: execv child1\n", 20);
        exit(1);
    }

    pid_t pid2 = fork();
    if (pid2 == -1) {
        write(2, "Error: fork child2\n", 19);
        exit(1);
    }

    if (pid2 == 0) {
        dup2(pipe2[0], 0);
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        char path[2048];
        snprintf(path, sizeof(path) - 1, "%s/child", progpath);

        char *args[] = {"./child", file2, NULL};
        execv(path, args);
        write(2, "Error: execv child2\n", 20);
        exit(1);
```

```
    }

    close(pipe1[0]);
    close(pipe2[0]);

    char buf[2048];
    ssize_t bytes;
    int pos = 0;

    while ((bytes = read(input_fd, buf + pos, sizeof(buf) - pos - 1)) > 0) {
        bytes += pos;
        pos = 0;

        for (ssize_t i = 0; i < bytes; i++) {
            if (buf[i] == '\n') {
                int len = i + 1;

                if (rand() % 100 < 80)
                    write(pipe1[1], buf + pos, len - pos);
                else
                    write(pipe2[1], buf + pos, len - pos);

                pos = len;
            }
        }

        if (pos < bytes) {
            int remaining = bytes - pos;
            for(int i = 0; i < remaining; i++)
                buf[i] = buf[pos + i];
            pos = remaining;
        } else
            pos = 0;
    }

    if (pos > 0) {
    if (rand() % 100 < 80)
        write(pipe1[1], buf, pos);
    else
        write(pipe2[1], buf, pos);
    }

    close(input_fd);
    close(pipe1[1]);
    close(pipe2[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

child.c

```c
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

int is_vowel(char c) {
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' || c == 'y' ||
           c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U' || c == 'Y';
}

int main(int argc, char **argv) {
    if (argc < 2) {
        write(2, "Error: no filename\n", 19);
        exit(1);
    }

    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        write(2, "Error open\n", 12);
        exit(1);
    }

    char buf[2048];
    ssize_t bytes;
    while ((bytes = read(0, buf, sizeof(buf))) > 0) {
        char out[1024];
        int j = 0;
        for (int i = 0; i < bytes; i++) {
            if (!is_vowel(buf[i]))
                out[j++] = buf[i];
        }

        write(fd, out, j);
    }

    close(fd);
    return 0;
}
```

Протокол работы программы

Тестирование:

>>> cat < test1.txt

<<< Hello world

My name is Nikita

This is a Operating Systems

I try to do this lab

>>> cat < test2.txt

<<< It's interesting TesT?

Naaah... 12IYha

``eiwoePs

sKEOo;a

dertyots werff vrgrtu7ui

>>> cat < test3.txt

<<< qwqwqwqwqwqwqw

trtrtrtrtrtrtr

kkkllkllllkkkk

mnmnmnmnmnmnn

_____---------_____

Test 1: >>> gcc -o parent parent.c

gcc -o child child.c

./parent test1.txt file1.txt file2.txt

cat < file1.txt

<<< Hll wrld

Ths s prtng Sstms

tr t d ths lb

>>> cat < file2.txt

<<< M nm s Nkt

Test 2: >>> gcc -o parent parent.c

gcc -o child child.c

./parent test2.txt file1.txt file2.txt

cat < file1.txt

<<< sK;

>>> cat < file2.txt

<<< t's ntrstng TsT?

Nh... 12h

``wPs

Test 3: >>> gcc -o parent parent.c

gcc -o child child.c

./parent test3.txt file1.txt file2.txt

cat < file1.txt

<<< qwqwqwqwqwqwqw

trtrtrtrtrtrtr

mnmnmnmnmnmnn

_____---------_____

>>> cat < file2.txt

<<< kkkllkllllkkkk

Вывод

В ходе лабораторной работы были изучены системные вызовы для создания процессов (fork, execv), организации межпроцессорного взаимодействия (pipe, dup2) и работы с файлами (open, read, write, close). Разработана программа, в которой родительский процесс создает двух дочерних, фильтрует строки из входного файла по вероятностному правилу 80/20 и передает их через каналы. Дочерние процессы удаляют гласные и записывают результат в отдельные файлы.